

Manual de Usuario

K-QGMIP

Extensión a k-Particiones de GeoMIP y QNodes

Universidad de Caldas

Facultad de Ingeniería · Análisis y Diseño de Algoritmos

2026-1

Estudiantes:

Cristian David González

Daniel Felipe Franco

Docente:

Luz Enith Guerrero Mendieta

15 de junio de 2026

Índice

1. Introducción	3
2. ¿Qué es el software y para qué sirve?	4
3. Requisitos del usuario	5
3.1. Requisitos de software	5
3.2. Requisitos de hardware	6
3.3. Requisitos de conocimiento	7
4. Instalación y configuración	7
4.1. Paso 1 — Obtener el código fuente	7
4.2. Paso 2 — Verificar Python 3.12	7
4.3. Paso 3 — Crear el entorno virtual	8
4.4. Paso 4 — Activar el entorno virtual	8
4.5. Paso 5 — Instalar las dependencias	8
4.6. Paso 6 — Verificar la instalación	9
4.7. Nota sobre paralelización	9
5. Uso básico del software	9
5.1. Flujo general de uso	9
5.2. Parámetros que debe especificar el usuario	10
5.3. Formas de proporcionar los parámetros	11
6. Ejecución del software	11
6.1. Entrada de datos del sistema	12
6.1.1. Matriz de Probabilidad de Transición (TPM)	12
6.1.2. Estado inicial del sistema	13
6.1.3. Máscaras binarias	13
6.1.4. Inicialización y preparación del subsistema	13
6.2. Método KGeoMIP (Estrategia Geométrica)	13
6.2.1. Parámetros de configuración	14
6.2.2. Pasos de ejecución	14
6.2.3. Ejemplo de ejecución	15
6.3. Método KQNodes (Estrategia Submodular)	15
6.3.1. Parámetros de configuración	15
6.3.2. Pasos de ejecución	15
6.3.3. Ejemplo de ejecución	16
7. Interpretación de resultados	17
7.1. KQNodes — Estrategia submodular	17
7.1.1. Estructura de la solución	17
7.1.2. La partición óptima	17
7.1.3. El valor de pérdida φ	18

7.1.4.	Uso de los resultados	19
7.2.	KGeoMIP — Estrategia geométrica	20
7.2.1.	Estructura de la solución	20
7.2.2.	La partición óptima y el valor φ	20
7.2.3.	Diferencias respecto a KQNodes	20

1. Introducción

K-QGMIP es un *framework* de análisis computacional que permite identificar la **Partición de Mínima Información de orden k (k -MIP)** en sistemas causales discretos, en el marco de la **Teoría de la Información Integrada versión 4.0 (IIT 4.0)**. El *framework* extiende las estrategias GeoMIP y QNodes —desarrolladas originalmente para biparticiones ($k = 2$)— al caso general de k -particiones con $k \in \{2, 3, 4, 5\}$, implementando dos métodos complementarios:

- **KGeoMIP**: estrategia geométrica que aprovecha la representación del espacio de estados como un hipercubo n -dimensional y reutiliza la tabla de costos de transiciones ya calculada para identificar particiones óptimas sin búsqueda exhaustiva.
- **KQNodes**: estrategia submodular que extiende el algoritmo de Queyranne (Ordenamiento de Máxima Adyacencia) a k particiones, reduciendo la complejidad combinatoria de $\mathcal{O}(S(n, k))$ evaluaciones exhaustivas a una fracción manejable.

Desde la perspectiva del usuario, **K-QGMIP** recibe como entrada la Matriz de Probabilidad de Transición (TPM) de un sistema, el estado inicial y las máscaras binarias que definen el subsistema de interés, y devuelve la k -partición óptima junto con su pérdida de información integrada φ . El usuario puede seleccionar el método de cálculo (KGeoMIP o KQNodes), el valor de k y ejecutar el análisis sobre sistemas de distintos tamaños, desde redes de validación de 10–15 nodos hasta sistemas de escala moderada con 20 o más nodos.

A quién va dirigido este manual

Este manual está dirigido a **investigadores, estudiantes y profesionales** que deseen utilizar K-QGMIP para analizar la estructura causal de sistemas discretos bajo el formalismo de IIT 4.0. Se asume que el lector posee:

- Conocimiento básico de **Python 3.x** y capacidad de ejecutar scripts desde línea de comandos o un entorno de desarrollo integrado (IDE).
- Familiaridad general con el concepto de **matriz de transición probabilística** y con la noción de información integrada (φ), sin necesidad de dominar sus fundamentos matemáticos avanzados.
- Acceso a una máquina con recursos acordes al tamaño del sistema a analizar (ver Sección de Requisitos del Usuario).

No se requieren conocimientos previos sobre la teoría de números de Stirling, la minimización de funciones submodulares ni la geometría de hipercubos; estos conceptos forman parte del diseño interno del *framework* y el usuario no necesita comprenderlos para obtener resultados.

Qué no es necesario conocer

Para utilizar K-QGMIP de forma efectiva, el usuario **no necesita**:

- Conocer el funcionamiento interno de las estrategias KGeoMIP o KQNodes (tabla de costos, Ordenamiento de Máxima Adyacencia, NCubos, etc.).
- Comprender la demostración de submodularidad de la función de pérdida φ ni el análisis de complejidad asintótica de los algoritmos.
- Modificar el código fuente ni configurar las dependencias del entorno de forma manual, más allá de los pasos de instalación descritos en la Sección de Instalación y Configuración.
- Conocer la arquitectura interna del proyecto (patrones de diseño, jerarquía de clases, estructura de módulos) salvo que desee extender o personalizar el *framework*.

El presente manual cubre exclusivamente el uso del sistema desde la perspectiva del usuario final. Para detalles sobre la arquitectura, los algoritmos y el análisis de complejidad, consulte el **Manual Técnico** del proyecto.

2. ¿Qué es el software y para qué sirve?

K-QGMIP es un *framework* computacional en Python diseñado para calcular la **k-Partición de Mínima Información** (*k-MIP*) de sistemas causales discretos. Dado un sistema de n variables binarias y su Matriz de Probabilidad de Transición (TPM), el *framework* identifica la forma óptima de dividir el sistema en k partes independientes de modo que la pérdida de información integrada φ sea mínima. Este resultado revela la **estructura modular** del sistema: las divisiones que menos perturban su dinámica causal son precisamente aquellas con menor información integrada.

¿Qué puede hacer el usuario con K-QGMIP?

Con K-QGMIP el usuario puede:

- **Seleccionar el número de particiones k :** analizar el sistema con $k \in \{2, 3, 4, 5\}$ partes, desde la bipartición clásica hasta divisiones en cinco subconjuntos independientes.
- **Elegir el método de cálculo:** el *framework* ofrece dos estrategias complementarias que el usuario puede seleccionar según el tamaño del sistema y el tiempo de cómputo disponible:
 - **KGeoMIP:** estrategia geométrica, adecuada para sistemas de tamaño moderado a grande. Reutiliza la tabla de costos de transiciones entre estados para evaluar candidatos de k -partición sin necesidad de búsqueda exhaustiva.
 - **KQNodes:** estrategia submodular basada en el algoritmo de Queyranne, con menor complejidad asintótica para valores bajos de k . Prioritaria para sistemas con muchos nodos donde la eficiencia es crítica.
- **Analizar sistemas de distintos tamaños:** desde redes pequeñas de validación (10–15 nodos) hasta sistemas de escala moderada con 20 o más nodos, cargando la

TPM desde un archivo CSV estándar.

- **Obtener la partición óptima con su pérdida:** el resultado incluye la k -partición identificada, el valor de φ asociado y metadatos del subsistema evaluado (estado inicial, máscaras binarias, tiempo de ejecución).
- **Ejecutar análisis en lote:** procesar múltiples subsistemas definidos en un archivo Excel, obteniendo resultados comparativos en formato CSV para análisis posterior.

Contexto de uso

K-QGMIP está orientado a tres contextos principales:

1. **Investigación académica en IIT 4.0:** el *framework* permite estudiar cómo varía la estructura de mínima información de un sistema al aumentar el número de particiones permitidas k , aportando evidencia empírica sobre la organización modular de sistemas causales.
2. **Experimentos exploratorios:** el usuario puede probar rápidamente distintas combinaciones de método, k y subsistema para identificar configuraciones de interés antes de lanzar análisis sistemáticos de mayor costo computacional.
3. **Análisis sistemáticos reproducibles:** la semilla aleatoria fija (`numpy seed = 73`) y la ejecución desde archivos de configuración garantizan que los resultados son deterministas y reproducibles entre sesiones y máquinas.

3. Requisitos del usuario

Esta sección lista los prerequisites de software, hardware y conocimiento necesarios para ejecutar K-QGMIP. Cumplirlos garantiza resultados correctos y tiempos de ejecución razonables.

3.1. Requisitos de software

- **Python 3.12 o superior.** El proyecto declara `requires-python = ">=3.12"` en su configuración. Versiones anteriores no están soportadas.
- **Gestor de dependencias uv.** Es el gestor recomendado por el proyecto para crear el entorno virtual e instalar todas las bibliotecas en una sola operación (ver Sección de Instalación y Configuración). Como alternativa puede usarse `pip` con el archivo `pyproject.toml`.
- **Dependencias declaradas en `pyproject.toml`.** La tabla siguiente resume las bibliotecas requeridas:

Paquete	Versión mínima	Uso en K-QGMIP
numpy	$\geq 2.0.2$	Operaciones matriciales, TPM, N-Cubos
scipy	$\geq 1.17.0$	Earth Mover's Distance (EMD) entre distribuciones
pandas	$\geq 2.3.3$	Lectura de Excel y escritura de resultados en CSV
openpyxl	$\geq 3.1.5$	Motor de lectura/escritura de archivos Excel
pyphi	$\geq 1.2.0$	Referencia de validación para casos pequeños ($k = 2$)
pyinstrument	$\geq 5.1.2$	Generación de reportes de rendimiento (HTML)
colorama	$\geq 0.4.6$	Visualización colorizada de resultados en consola
pyttsx3	≥ 2.99	Síntesis de voz al encontrar la solución óptima

Cuadro 1: Dependencias externas requeridas por K-QGMIP.

3.2. Requisitos de hardware

El costo computacional de K-QGMIP escala con dos factores independientes: el tamaño del sistema (n) y el número de particiones (k). La Tabla 2 orienta al usuario sobre los recursos mínimos recomendados según el rango de n ; el impacto de k se discute en la nota al pie de la tabla.

Tamaño del sistema	RAM mínima	Perfil de máquina recomendado
$n \leq 6$	4 GB	Cualquier computador moderno (portátil académico básico)
$n = 8-10$	4 GB o superior	Portátil o escritorio estándar de uso académico
$n = 15-17$	16 GB	PC de escritorio con procesador de al menos 4 núcleos
$n = 20-22$	32 GB	PC de alto rendimiento
$n \geq 25$	32 GB o superior	PC de alto rendimiento

Cuadro 2: Recursos de hardware recomendados según tamaño del sistema analizado ($k = 2$).

Efecto de k sobre el costo computacional: los recursos de la Tabla 2 aplican al caso base $k = 2$. Al incrementar k , lo que puede aumentar considerablemente el tiempo de ejecución y el uso de memoria para un mismo n . Como regla general: a mayor k , mayor costo; se recomienda comenzar siempre con $k = 2$ para estimar la carga computacional del sistema antes de escalar a valores superiores.

GPU: no es un requisito obligatorio. K-QGMIP corre íntegramente en CPU. Sin embargo, si el entorno dispone de múltiples núcleos, la paralelización de evaluaciones de particiones puede configurarse para aprovecharlos y reducir los tiempos indicados.

3.3. Requisitos de conocimiento

Para utilizar K-QGMIP no se exige experiencia en algoritmos avanzados ni en matemáticas de IIT 4.0, pero sí se asume la siguiente familiaridad básica:

- **Python básico:** capacidad de ejecutar un script (`python main.py`), instalar paquetes y leer mensajes de error de la consola.
- **Línea de comandos o IDE:** saber navegar directorios y abrir un terminal (símbolo del sistema, PowerShell o terminal de VS Code / PyCharm).
- **Formato de datos:** entender qué es un archivo CSV y un archivo Excel (`.xlsx`), puesto que la TPM se carga desde CSV y los casos de prueba en lote se definen en Excel.
- **Nociones de IIT** (*opcional pero recomendado*): familiaridad con los conceptos de matriz de transición probabilística, estado inicial del sistema y la idea general de información integrada φ , para poder interpretar correctamente los resultados obtenidos.

4. Instalación y configuración

K-QGMIP se distribuye como **código fuente** a través de un repositorio Git. No existe un instalador ejecutable: el usuario descarga el repositorio, crea un entorno virtual de Python y ejecuta la instalación de dependencias con un único comando.

4.1. Paso 1 — Obtener el código fuente

Clone el repositorio con Git o descargue el archivo ZIP desde la plataforma de distribución del proyecto:

```
git clone https://github.com/CristianDavGon/algoritmos_final1.git
```

```
cd "Proyecto_V03_FINAL"
```

Si no tiene Git instalado, puede descargar el ZIP directamente desde la interfaz web del repositorio y descomprimirlo en la ruta de su preferencia.

4.2. Paso 2 — Verificar Python 3.12

Confirme que tiene Python 3.12 o superior instalado:

```
python --version
```

La salida debe mostrar `Python 3.12.x` o superior. Si no cumple este requisito, descargue e instale la versión correcta desde `python.org` antes de continuar.

4.3. Paso 3 — Crear el entorno virtual

Se recomienda usar `uv`, el gestor de entornos del proyecto, para aislar las dependencias y evitar conflictos con otros proyectos Python en el sistema.

Con `uv` (recomendado):

```
pip install uv
cd code
uv venv
```

Con `venv` estándar (alternativa):

```
cd code
python -m venv .venv
```

Esto crea una carpeta `.venv/` dentro de `code/` con un intérprete Python aislado. No modifique ni elimine esta carpeta manualmente.

4.4. Paso 4 — Activar el entorno virtual

Antes de instalar dependencias o ejecutar cualquier script, active el entorno:

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Windows (símbolo del sistema):

```
.venv\Scripts\activate.bat
```

Linux / macOS:

```
source .venv/bin/activate
```

El prefijo (`.venv`) en el prompt del terminal confirma que el entorno está activo. Recuerde activarlo cada vez que abra una nueva sesión de terminal.

4.5. Paso 5 — Instalar las dependencias

Con el entorno activo, instale todas las dependencias declaradas en `pyproject.toml`:

Con `uv` (recomendado):

```
uv sync
```

Con `pip` (alternativa):

```
pip install -e .
```

`uv sync` resuelve las versiones exactas desde el archivo `uv.lock`, garantizando reproducibilidad. `pip install -e .` instala en modo editable y resuelve versiones mínimas desde `pyproject.toml`.

4.6. Paso 6 — Verificar la instalación

Compruebe que las bibliotecas principales están disponibles ejecutando desde la carpeta `code/`:

```
python -c "import numpy, scipy, pandas, pyphi; print('OK')"
```

Si la salida es `OK`, el entorno está listo para ejecutar K-QGMIP. Si algún paquete falla, repita el Paso 5 asegurándose de que el entorno virtual esté activo.

4.7. Nota sobre paralelización

K-QGMIP corre íntegramente sobre **CPU** sin configuración adicional. Si la máquina dispone de múltiples núcleos, el *framework* puede aprovecharlos para paralelizar la evaluación de particiones candidatas; en ese caso no se requiere ningún hardware especializado (GPU) ni ajustes manuales: el sistema detecta automáticamente los núcleos disponibles y los utiliza durante la ejecución.

5. Uso básico del software

El flujo de uso de K-QGMIP es el mismo independientemente del método elegido (KQNodes o KGeoMIP): el usuario edita un **script de entrada** dedicado a cada método, configura los parámetros del análisis y lo ejecuta desde el terminal. El *framework* se encarga de orquestar internamente la carga de la red, la preparación del subsistema, la ejecución del algoritmo y el guardado de resultados.

5.1. Flujo general de uso

1. **Seleccionar el método:** elegir entre KQNodes o KGeoMIP según el tamaño del sistema y los recursos disponibles (ver Sección de Requisitos).
2. **Abrir el script de entrada del método elegido:**
 - KQNodes: `code/QNodes/exec_kqnodes.py`
 - KGeoMIP: `code/GeoMIP/exec_kgeomip.py`
3. **Configurar los parámetros** directamente en el script (detallados en la subsección siguiente).
4. **Ejecutar el script** desde la carpeta correspondiente:

```
# Para KQNodes
cd code/QNodes
```

```

uv run exec_kqnodes.py

# Para KGeoMIP
cd code/GeoMIP
uv run exec_kgeomip.py

```

5. **Revisar los resultados** generados automáticamente en `results/{metodo}/` en formato CSV y Markdown.

5.2. Parámetros que debe especificar el usuario

Cada script de entrada expone cuatro parámetros al inicio del archivo que el usuario debe ajustar antes de ejecutar. Tres de ellos son comunes a ambos métodos (ESTADO, K, MUESTRA); el cuarto difiere según el método.

KQNodes — `code/QNodes/exec_kqnodes.py`

Parámetro	Ejemplo	Descripción
ESTADO	"1" + "0" * 19	Estado inicial $s(t)$: cadena binaria de longitud n . Define implícitamente el tamaño del sistema.
K	5	Número de partes de la k-partición. Valores válidos: 2, 3, 4, 5. Con 2 reproduce la bipartición clásica.
CRITERIO	"C4"	Criterio de refinamiento submodular. "C4" = corte marginal mínimo (recomendado); "C1" = tamaño máximo de parte.
MUESTRA	"A"	Letra de la red de prueba ($N\{n\}\{MUESTRA\}.csv$).

Cuadro 3: Parámetros de configuración de KQNodes en `exec_kqnodes.py`.

Parámetro	Ejemplo	Descripción
ESTADO	"1" + "0" * 24	Estado inicial $s(t)$: cadena binaria de longitud n . Define implícitamente el tamaño del sistema.
K	5	Número de partes de la k -partición. Valores válidos: 2, 3, 4, 5. Con 2 reproduce exactamente GeoMIP.
VARIANTE	"E4"	Heurística de refinamiento geométrico. "E4" = divisivo por MinHeap con $\Delta\Phi$ (recomendado; garantiza regresión $k = 2$ y monotonidad); "A" = clustering aglomerativo (solo para comparación experimental).
MUESTRA	"A"	Letra de la red de prueba ($N\{n\}\{MUESTRA\}.csv$).

Cuadro 4: Parámetros de configuración de KGeoMIP en `exec_kgeomip.py`.

Las máscaras binarias de **condición**, **alcance** y **mecanismo** no se configuran manualmente en los scripts: se leen automáticamente de las hojas del archivo Excel de pruebas (`data/DatosPruebas2026_1.xlsx`), donde están expresadas como letras ("ABCDE") y el *framework* las convierte a cadenas binarias en tiempo de ejecución.

5.3. Formas de proporcionar los parámetros

K-QGMIP admite dos formas de entrada de datos:

- **Directamente en el script** (análisis individual): el usuario edita ESTADO, K, CRITERIO y MUESTRA en el script de entrada y ejecuta una sola corrida. Útil para explorar un sistema específico o depurar resultados.
- **Mediante archivo Excel** (análisis en lote): el *framework* lee todas las combinaciones de alcance y mecanismo definidas en `DatosPruebas2026_1.xlsx` para el tamaño n correspondiente, y ejecuta el método sobre cada una. El usuario solo configura ESTADO, K, CRITERIO y MUESTRA; el resto lo gestiona el *framework* automáticamente. Esta es la modalidad de ejecución por defecto al correr los scripts de entrada.

6. Ejecución del software

Esta sección describe en detalle cómo ejecutar K-QGMIP. Se organiza en tres partes: la entrada de datos del sistema (común a ambos métodos), la ejecución del método KGeoMIP

y la ejecución del método KQNodes. Antes de proceder, asegúrese de haber completado la instalación descrita en la Sección de Instalación y Configuración.

6.1. Entrada de datos del sistema

Ambos métodos requieren los mismos datos de entrada: una Matriz de Probabilidad de Transición (TPM), un estado inicial y tres máscaras binarias que definen el subsistema de interés.

6.1.1. Matriz de Probabilidad de Transición (TPM)

La **TPM** codifica la dinámica del sistema: para cada estado $s(t)$ del sistema en el instante t , la TPM especifica la distribución de probabilidad sobre los estados posibles en $t + 1$. K-QGMIP utiliza la representación *estado-nodo* en notación *little-endian*.

Las redes de prueba se distribuyen como archivos CSV con el esquema de nombre:

`N{n}{pagina}.csv`

donde **n** es el número de nodos y **pagina** es una letra que distingue distintas redes del mismo tamaño (A, B, etc.). Por ejemplo, `N20A.csv` es la red de muestra de 20 nodos, página A.

La ubicación de los archivos depende del método utilizado:

Método	Carpeta de muestras
KQNodes	<code>code/QNodes/src/.samples/</code>
KGeoMIP	<code>code/GeoMIP/data/samples/</code>

Cuadro 5: Ubicación de las redes de prueba (TPM en CSV) por método.

En KGeoMIP, el **Manager** busca la red en `data/samples/` de forma predeterminada; la ruta puede sobrescribirse con la variable de entorno `GEOMIP_SAMPLES_DIR` si se desea apuntar a un directorio distinto.

La TPM se carga automáticamente a través de la clase **Manager**:

```
import numpy as np
from src.controllers.manager import Manager

gestor = Manager("10000000000000000000") # estado inicial n=20
tpm = gestor.cargar_red() # np.ndarray de shape (2^n, n)
```

Internamente, `cargar_red()` usa `numpy.genfromtxt` con delimitador `;` y selecciona el archivo `N{n}{pagina}.csv` correspondiente al estado inicial configurado.

6.1.2. Estado inicial del sistema

El **estado inicial** $s(t)$ es una cadena binaria de longitud n que indica el estado de cada nodo en el instante de análisis. El nodo más a la izquierda corresponde al bit de menor peso (*little-endian*).

```
ESTADO = "10000000000000000000" # n=20: nodo 0 activo, resto inactivos
ESTADO = "11111111"              # n=8: todos los nodos activos
ESTADO = "101"                   # n=3: nodos 0 y 2 activos
```

La longitud de la cadena determina automáticamente el tamaño n del sistema.

6.1.3. Máscaras binarias

Tres cadenas binarias de longitud n definen el subsistema que se analizará:

- **Condición:** nodos cuyo estado en t se fija al analizar el subsistema. Habitualmente se establece como "1"* n (todos los nodos).
- **Alcance** (*scope*): subconjunto de nodos en $t + 1$ que se incluyen en el análisis. Ejemplos para $n = 5$: "11100" incluye los nodos A, B y C.
- **Mecanismo:** subconjunto de nodos en t que actúan como causa del subsistema. Ejemplos para $n = 5$: "00111" incluye los nodos C, D y E.

En los archivos de prueba por lotes (Excel), alcance y mecanismo se especifican con letras ("ABC", "CDE") y el *framework* los convierte automáticamente a cadenas binarias.

6.1.4. Inicialización y preparación del subsistema

El objeto **Manager** centraliza la carga de la TPM y la configuración de rutas. Una vez creado, se pasa la TPM directamente al constructor de la estrategia elegida:

```
from src.controllers.manager import Manager
from src.strategies.kqnodes import KQNodes

gestor = Manager(ESTADO)
tpm     = gestor.cargar_red()
kqn     = KQNodes(tpm)
```

La preparación del subsistema (condicionamiento y substracción) es automática: ocurre dentro del método `aplicar_estrategia()` de la clase base **SIA** y el usuario no interviene directamente en ese proceso.

6.2. Método KGeoMIP (Estrategia Geométrica)

KGeoMIP extiende la estrategia geométrica de GeoMIP al caso de k -particiones mediante un **refinamiento divisivo top-down** (variante E4): parte de la bipartición óptima que GeoMIP ya encontró ($k = 2$) y en cada paso divide iterativamente la parte cuyo corte produce el menor incremento de pérdida $\Delta\Phi$, hasta obtener k partes. Toda la tabla de

costos T y la matriz de similitud causal S se calculan **una sola vez** y se reutilizan para todos los valores de k , lo que hace al método eficiente incluso para sistemas de hasta 25 nodos.

6.2.1. Parámetros de configuración

Todos los parámetros se configuran en el archivo `code/GeoMIP/exec_kgeomip.py`:

```
ESTADO: str = "1" + "0" * 24 # estado inicial (define n=25)
K:      int = 5               # numero de partes (2, 3, 4 o 5)
VARIANTE: str = "E4"         # heurística de refinamiento
MUESTRA: str = "A"           # página de la red de prueba
```

Parámetro	Valores válidos	Descripción
ESTADO	Cadena binaria longitud n	Estado inicial $s(t)$. Su longitud determina el tamaño del sistema y la red a cargar (<code>N{n}{MUESTRA}.csv</code>).
K	2, 3, 4, 5	Número de partes de la k -partición. Con $K=2$ reproduce exactamente el resultado de GeoMIP.
VARIANTE	"E4", "A"	"E4" = refinamiento divisivo por MinHeap con $\Delta\Phi$ (recomendado; garantiza regresión $k = 2$ y monotonicidad). "A" = clustering aglomerativo sobre la matriz de similitud S (solo para comparación experimental, sin garantías).
MUESTRA	"A", "B", ...	Letra que identifica la red de prueba.

Cuadro 6: Parámetros configurables de KGeoMIP en `exec_kgeomip.py`.

6.2.2. Pasos de ejecución

1. **Abrir el archivo de configuración.** Con el entorno virtual activo, navegue a la carpeta del método:

```
cd code/GeoMIP
```

2. **Configurar los parámetros.** Edite ESTADO, K, VARIANTE y MUESTRA en `exec_kgeomip.py` según el sistema y el análisis deseado.

3. **Ejecutar el script.**

```
uv run exec_kgeomip.py
```

El *framework* carga la TPM desde `data/samples/N{n}{MUESTRA}.csv`, lee todas las filas de alcance y mecanismo del Excel de pruebas para el tamaño n , y ejecuta KGeoMIP sobre cada combinación con los parámetros configurados.

4. **Localizar los resultados.** Los archivos de salida se guardan en:

```
results/kgeomip/resultado__N{n}__MUESTRA_{K}.csv  
results/kgeomip/resultado__N{n}__MUESTRA_{K}.md
```

El CSV contiene una fila por cada combinación (prueba, k , variante) con las columnas: Prueba, Alcance, Mecanismo, k , Criterio, Partición, Pérdida (φ), Tiempo (s).

6.2.3. Ejemplo de ejecución

El siguiente ejemplo ejecuta KGeoMIP sobre la red de 25 nodos (muestra A), buscando la 5-partición de mínima información con la variante E4:

```
# exec_kgeomip.py  
ESTADO = "1" + "0" * 24 # n=25, nodo 0 activo  
K       = 5  
VARIANTE = "E4"  
MUESTRA = "A"
```

Salida esperada en consola:

```
n=25: 45 pruebas x variante E4  
Prueba 1/ 45 -- Alc: ABCDE Mec: ABCDE  
Prueba 2/ 45 -- Alc: ABCDEF Mec: ABCDE  
...  
CSV: results/kgeomip/resultado__N25_A_5.csv (45 filas)  
MD: results/kgeomip/resultado__N25_A_5.md
```

6.3. Método KQNodes (Estrategia Submodular)

KQNodes extiende el algoritmo de Queyranne (Ordenamiento de Máxima Adyacencia) al caso de k -particiones mediante refinamiento iterativo de biparticiones. Es la estrategia prioritaria para sistemas de tamaño moderado a grande donde la eficiencia computacional es crítica.

6.3.1. Parámetros de configuración

Todos los parámetros se configuran directamente en el archivo `code/QNodes/exec_kqnodes.py`:

```
ESTADO: str = "1" + "0" * 19 # estado inicial (define n=20)  
K:      int = 5                # numero de partes (2, 3, 4 o 5)  
CRITERIO: str = "C4"          # criterio de refinamiento  
MUESTRA: str = "A"            # pagina de la red de prueba
```

6.3.2. Pasos de ejecución

1. **Abrir el archivo de configuración.** En un terminal con el entorno virtual activo, navegue a la carpeta del método:

Parámetro	Valores válidos	Descripción
ESTADO	Cadena binaria de longitud n	Estado inicial $s(t)$. Su longitud determina el tamaño del sistema.
K	2, 3, 4, 5	Número de partes de la k-partición.
CRITERIO	"C4", "C1"	Criterio de refinamiento: C4 = corte marginal mínimo; C1 = tamaño máximo de parte.
MUESTRA	"A", "B", ...	Letra de la red de prueba a analizar ($N\{n\}\{MUESTRA\}.csv$).

Cuadro 7: Parámetros configurables de KQNodes en `exec_kqnodes.py`.

```
cd code/QNodes
```

2. **Configurar los parámetros.** Edite las cuatro variables al inicio de `exec_kqnodes.py` según el sistema y el análisis que desea realizar.

3. **Ejecutar el script.**

```
uv run exec_kqnodes.py
```

El *framework* carga automáticamente la TPM correspondiente desde `data/DatosPruebas2026_1.x` procesa todas las filas de alcance y mecanismo de la hoja asociada al tamaño n , y ejecuta KQNodes para cada combinación con el k y criterio configurados.

4. **Localizar los resultados.** Los archivos de salida se guardan en:

```
results/kqnodes/resultado__N{n}_{MUESTRA}_{K}.csv
results/kqnodes/resultado__N{n}_{MUESTRA}_{K}.md
```

El CSV contiene una fila por cada combinación (prueba, k , criterio) con las columnas: Prueba, Alcance, Mecanismo, k , Criterio, Partición, Pérdida (φ), Tiempo (s).

6.3.3. Ejemplo de ejecución

El siguiente ejemplo ejecuta KQNodes sobre la red de 20 nodos (muestra A), buscando la 5-partición de mínima información con criterio de corte marginal mínimo:

```
# exec_kqnodes.py
ESTADO = "1" + "0" * 19 # n=20, nodo 0 activo
K       = 5
CRITERIO = "C4"
MUESTRA = "A"
```

Salida esperada en consola:

```
n=20: 45 pruebas x 1 k x 1 criterios = 45 ejecuciones
Prueba 1/ 45 -- Alc: ABCDE Mec: ABCDE
Prueba 2/ 45 -- Alc: ABCDEF Mec: ABCDE
...
```

CSV: `results/kqnodes/resultado__N20_A_5.csv` (45 filas)
 MD: `results/kqnodes/resultado__N20_A_5.md`

7. Interpretación de resultados

Una vez ejecutado el *framework*, los resultados se presentan en dos formatos complementarios: un archivo CSV con una fila por caso analizado y un archivo Markdown con tabla de resumen y detalle por caso. Esta sección explica cómo leer e interpretar cada campo de la solución, qué significa la partición obtenida y qué cuantifica el valor de pérdida φ .

7.1. KQNodes — Estrategia submodular

7.1.1. Estructura de la solución

Cada caso analizado produce un objeto **Solution** con los siguientes campos, que se reflejan directamente en las columnas del CSV y del Markdown de resultados:

Campo	Tipo	Descripción
Prueba	Entero	Número de fila del Excel de pruebas (1-indexado).
Alcance	Letras	Nodos del subsistema en $t + 1$ (p.ej. ABCDE).
Mecanismo	Letras	Nodos del subsistema en t (p.ej. ACE).
k	Entero	Número de partes solicitado.
Criterio	Cadena	Heurística usada (C4 o C1).
Partición	Texto	Agrupación óptima de nodos en k partes.
Pérdida (φ)	Real	Información integrada perdida al aplicar la partición.
Tiempo (s)	Real	Tiempo de cómputo en segundos para ese caso.

Cuadro 8: Columnas del CSV y Markdown de resultados de KQNodes.

Adicionalmente, el objeto interno **Solution** contiene dos distribuciones de probabilidad que no se exportan al CSV pero están disponibles para análisis programático:

- **distribucion_subsistema**: vector de probabilidades marginales del subsistema original (sin partir), de forma $(N,)$.
- **distribucion_particion**: vector de probabilidades del subsistema tratando las k partes como independientes.

7.1.2. La partición óptima

La **partición** representa la división del subsistema en k grupos de nodos que produce la menor pérdida de información causal. Su formato en el archivo de resultados es:

(Nodos_parte_1, dims_parte_1) | (Nodos_parte_2, dims_parte_2) | ...

Cada grupo (X, Y) indica los nodos involucrados; el símbolo $\emptyset$ aparece cuando la parte no tiene dimensiones de mecanismo asociadas (partición solo en el alcance, tiempo $t + 1$).

Ejemplo — Red de 8 nodos, $k = 5$, criterio C4. El caso #1 (alcance y mecanismo completos: ABCDEFGH) produce:

Particion: (A,0) | (B,0) | (C,0) | (D,0) | (E,F,G,H,0)

Perdida (ϕ): 4.000000

Tiempo: 0.0082 s

Interpretación: KQNodes divide el subsistema en 5 partes. Las primeras cuatro partes son singletons (A, B, C, D aislados), mientras que la quinta agrupa los nodos E, F, G y H. Esta es la forma de cortar el sistema que minimiza el incremento de φ respecto a la distribución original.

El caso #41 (alcance ACE, mecanismo ACE) produce:

Particion: (A,0) | (C,0) | (E,0)

Perdida (ϕ): 0.000000

Tiempo: 0.0016 s

Interpretación: $\varphi = 0$ indica **partición natural**: el subsistema {A, C, E} ya era causalmente independiente; cortarlo no produce pérdida alguna de información integrada. El subsistema no tiene información causal irreducible.

7.1.3. El valor de pérdida φ

El valor φ (pérdida) cuantifica la **diferencia entre la distribución del subsistema original y la distribución reconstruida** asumiendo que las k partes son **causalmente independientes**. Formalmente:

$$\varphi^* = \text{EMD} \left(p(s_{t+1}), \bigotimes_{i=1}^k p_i(s_{t+1}^{(i)}) \right) \quad (1)$$

donde EMD es la *Earth Mover's Distance* (en la variante efecto del proyecto: suma de diferencias absolutas de distribuciones marginales) y $\otimes$ denota el producto de distribuciones independientes de las k partes.

Propiedades importantes del valor φ :

- **Monotonidad:** $\varphi(k+1) \geq \varphi(k)$. Aumentar el número de partes solo puede mantener o aumentar la pérdida; nunca la reduce.
- **Cota inferior:** $\varphi \geq 0$ siempre (la EMD es no negativa).
- **Heurística:** para $k \geq 3$, KQNodes garantiza encontrar una k -partición de bajo φ (corte marginal mínimo), pero no necesariamente el mínimo global.

Valor de φ	Significado
$\varphi = 0$	Partición natural: las partes ya son causalmente independientes. El sistema es completamente reducible para ese subsistema.
$\varphi > 0$	El sistema tiene información causal irreducible: las partes interactúan causalmente y no pueden tratarse como independientes sin pérdida de información. A mayor φ , mayor integración causal.

Cuadro 9: Interpretación del valor de pérdida φ en KQNodes.

Métrica	Valor
Total de pruebas	49
φ promedio	1,80
φ máxima	4,00
φ mínima	0,00
Tiempo promedio	0,0038 s
Tiempo total	0,1869 s

Cuadro 10: Resumen de resultados de KQNodes para la red N8A con $k = 5$, criterio C4.

Resumen estadístico del experimento N8A, $k = 5$:

7.1.4. Uso de los resultados

Los archivos de salida generados por KQNodes se pueden utilizar de las siguientes formas:

- **Análisis estadístico:** el CSV puede importarse directamente en herramientas como pandas (Python), R o Excel para calcular distribuciones de φ , tiempos por tamaño de subsistema o comparar criterios C4 y C1.

```
import pandas as pd
```

```
df = pd.read_csv("results/kqnodes/resultado__N8_A_5.csv")
print(df["Pérdida_φ"].describe())
print(df[df["Pérdida_φ"] == 0]) # casos con particion natural
```

- **Comparación entre valores de k :** ejecutar KQNodes para $k \in \{2, 3, 4, 5\}$ sobre la misma red y comparar cómo evoluciona $\varphi(k)$, aprovechando la propiedad de monotonía para validar coherencia de los resultados.
- **Identificación de subsistemas irreducibles:** filtrar casos con $\varphi = 0$ para detectar subsistemas que el sistema puede tratar como partes independientes, lo que tiene interpretación directa en IIT 4.0 como ausencia de información causal integrada.
- **Exportación al manual técnico:** las tablas del Markdown generado (.md) están formateadas para incrustar directamente en informes o documentos de análisis.

7.2. KGeoMIP — Estrategia geométrica

7.2.1. Estructura de la solución

La estructura de resultados de KGeoMIP es idéntica a la de KQNodes; las columnas del CSV y del Markdown de salida son las mismas:

Campo	Tipo	Descripción
Prueba	Entero	Número de fila del Excel de pruebas (1-indexado).
Alcance	Letras	Nodos del subsistema en $t + 1$ (p.ej. ABCDE).
Mecanismo	Letras	Nodos del subsistema en t (p.ej. ACE).
k	Entero	Número de partes solicitado.
Criterio	Cadena	Variante usada (E4 o A).
Partición	Texto	Agrupación óptima de nodos en k partes.
Pérdida (φ)	Real	Información integrada perdida al aplicar la partición.
Tiempo (s)	Real	Tiempo de cómputo en segundos para ese caso.

Cuadro 11: Columnas del CSV y Markdown de resultados de KGeoMIP.

7.2.2. La partición óptima y el valor φ

La partición reportada por KGeoMIP se construye mediante refinamiento divisivo (**variante E4**): se parte de la bipartición óptima de GeoMIP ($k = 2$) y se divide iterativamente la parte cuyo corte produce el menor incremento $\Delta\Phi$, hasta alcanzar k partes. El resultado se escribe con el mismo formato que KQNodes:

```
Particion: (A,B,C | D,E,F | G,H | I | J)
Perdida (phi): 2.75
Tiempo: 0.043 s
```

El valor φ tiene la misma interpretación que en KQNodes (ver Sección 7.1): mide la distancia EMD entre la distribución original del subsistema y la distribución reconstruida asumiendo independencia causal entre las k partes.

Una propiedad específica de la variante E4 es la **garantía de regresión**: para $k = 2$, KGeoMIP reproduce exactamente el resultado de GeoMIP. Para $k \geq 3$, la secuencia de pérdidas es monótona no decreciente ($\varphi(k) \geq \varphi(k - 1)$).

7.2.3. Diferencias respecto a KQNodes

Aunque los formatos de salida y la interpretación de φ son equivalentes, los dos métodos difieren en la estrategia de búsqueda:

Aspecto	KQNodes	KGeoMIP
Fundamento	Función submodular de Queyranne	Tabla de costos geométrica sobre hipercubo n -dimensional
Estrategia	Ordenamiento de máxima adyacencia + refinamiento iterativo	Refinamiento divisivo top-down con MinHeap y $\Delta\Phi$
Garantía $k = 2$	Óptimo exacto (Queyranne)	Igual al óptimo de GeoMIP
Regresión $k > 2$	Heurística sin garantía de mínimo global	Monotonidad garantizada (E4)
Complejidad	$\mathcal{O}(n^3)$ por bipartición	$\mathcal{O}(n^2)$ por refinamiento (tabla precalculada)

Cuadro 12: Comparación entre KQNodes y KGeoMIP como herramientas de análisis.

Para un mismo subsistema, los valores de φ de ambos métodos **no tienen por qué ser iguales**: cada método encuentra una k -partición de bajo costo según su propia heurística, y la pérdida asociada puede diferir. La comparación sistemática entre métodos para los mismos casos de prueba es precisamente uno de los objetivos del experimento final del proyecto.